Faculty of Computing

BSc (Hons) in Software Engineering
Year 2 Semester 2 (2026)

SE2020 - Web and Mobile Technologies Lab 05

Practical Lab 05: Advanced HTML DOM Manipulation

1. **Scenario: Interactive Travel Registration Portal (Extended)**

   In Lab 01, you converted a static travel web page into an interactive page using basic DOM methods and events.

   In this lab, you will extend the same application by adding:

   - Advanced DOM traversal
   - Dynamic counters
   - Conditional styling
   - Input validation
   - Event delegation

   This lab focuses on improving user experience using DOM logic.

2. What You Will Learn

   By completing this lab, you will be able to:

   - Traverse the DOM using parent, child, and sibling relationships
   - Dynamically create and remove HTML elements
   - Apply conditional styling using JavaScript
   - Validate form inputs using DOM events
   - Handle events efficiently using event delegation

3. Starter Files

   Use the same files from Lab 01:

   - `index.html`
   - `script.js`

   - Do not change the existing HTML structure unless mentioned
   - All logic must be written inside `script.js`

4. DOM Traversal (New Concept)

   Task 1 – Access Elements Using DOM Relationships

   Add the following code to `script.js`:

```
let form = document.forms[0];
let title = document.getElementsByTagName("h1")[0];
let image = document.getElementById("travelImg");
let table = document.getElementById("travelerTable");

// ----------------------------
// Task 1: DOM Traversal
// ----------------------------
let formContainer = form.parentNode;      // parentNode usage
let inputs = form.children;               // children usage

for (let i = 0; i < inputs.length; i++) {
  // change background of all INPUT fields
  if (inputs[i].tagName === "INPUT") {
    inputs[i].style.backgroundColor = "#eef";
  }
  console.log("Child tag:", inputs[i].tagName);
}
```

   Explanation

   - **parentNode** accesses the parent element
   - **children** returns all child elements
   - Looping allows bulk DOM manipulation

5. Creating Dynamic Elements

   Task 2 – Display Total Registered Travelers

   Add the following code below existing DOM logic:

```
// Task 2: Dynamic traveler counter
// ----------------------------
let countPara = document.createElement("p");
countPara.id = "travelerCount";
countPara.innerText = "Total Registered Travelers: 0";
document.body.appendChild(countPara);

function updateCount() {
  // rows length includes header row, so -1
  let count = table.rows.length - 1;
  countPara.innerText = "Total Registered Travelers: " + count;
}
```

Call `updateCount()`:

- After adding a traveler
- After deleting a traveler

6. Conditional Styling Using DOM

   Task 3 – Highlight Rows Based on Age

   Update your `addTraveler()` function:

```
// Conditional styling based on age
if (age < 18) {
  row.style.backgroundColor = "#fff3cd";   // under 18
} else if (age >= 60) {
  row.style.backgroundColor = "#d1ecf1";   // seniors
}
```

   Explanation

   - DOM styles change dynamically
   - Conditions improve visual feedback

7. Real-Time Form Validation

   Task 4 – Validate Inputs While Typing

   Task 4.1 – Selecting Elements Using `querySelector()`

```
// Task 4: Real-time form validation
// ---------------------------
let submitBtn = document.querySelector("input[type='submit']");
let nameInput = document.getElementById("name");
let ageInput = document.getElementById("age");
let emailInput = document.getElementById("email");
```

   - `querySelector()` selects the first matching element
   - Uses CSS selector syntax
   - Can select:
     - By ID → `#id`
     - By type → `input`
     - By attribute → `input[type="submit"]`

Task 4.2 – Applying `querySelector()` in Form Validation

```
submitBtn.disabled = true;

function validateForm() {
  let valid = true;

  if (nameInput.value.trim() === "") {
    nameInput.style.border = "2px solid red";
    valid = false;
  } else {
    nameInput.style.border = "";
  }

  // simple validation for age
  let ageVal = parseInt(ageInput.value, 10);
  if (isNaN(ageVal) || ageVal <= 0) {
    ageInput.style.border = "2px solid red";
    valid = false;
  } else {
    ageInput.style.border = "";
  }

  if (!emailInput.value.includes("@")) {
    emailInput.style.border = "2px solid red";
    valid = false;
  } else {
    emailInput.style.border = "";
  }

  submitBtn.disabled = !valid;
}

nameInput.onkeyup = validateForm;
ageInput.onkeyup = validateForm;
emailInput.onkeyup = validateForm;
```

8. Dynamic Filtering Using DOM

   Task 5 – Filter Travelers by Age Group

   Add a dropdown in HTML above the table:

```
<select id="ageFilter">
  <option value="all">All</option>
  <option value="under18">Under 18</option>
  <option value="adult">18 - 59</option>
  </option>
</select>
```

Add the following code to `script.js`:

```javascript
// Task 5: Dynamic filtering by age group
// ---------------------------
document.getElementById("ageFilter").onchange = function () {
  let filter = this.value;
  let rows = table.rows;

  for (let i = 1; i < rows.length; i++) {
    let age = parseInt(rows[i].cells[1].innerText, 10);
    rows[i].style.display = "table-row";

    if (filter === "under18" && age >= 18) rows[i].style.display = "none";
    if (filter === "adult" && (age < 18 || age >= 60)) rows[i].style.display = "none";
    if (filter === "senior" && age < 60) rows[i].style.display = "none";
  }
};
```

9. Event Delegation (Best Practice)

   Task 6 – Handle Delete Buttons Efficiently

   Replace individual onclick handlers with:

```javascript
// Task 6: Event delegation for delete buttons
// ---------------------------
table.addEventListener("click", function (e) {
  if (e.target.tagName === "BUTTON") {
    let row = e.target.parentNode.parentNode;
    row.remove();
    updateCount();
  }
});
```

   Explanation

   - One listener handles all delete buttons
   - Improves performance and scalability

10. Expected Output

    After completing this lab:

    - Travelers are added and removed dynamically
    - Table rows change color based on age
    - Form validates inputs in real time
    - Traveler count updates automatically
    - Filtering works without page reload

11. Submission

    - Submit:
      - `index.html`
      - `script.js`
    - Follow naming conventions
    - Comment your JavaScript code